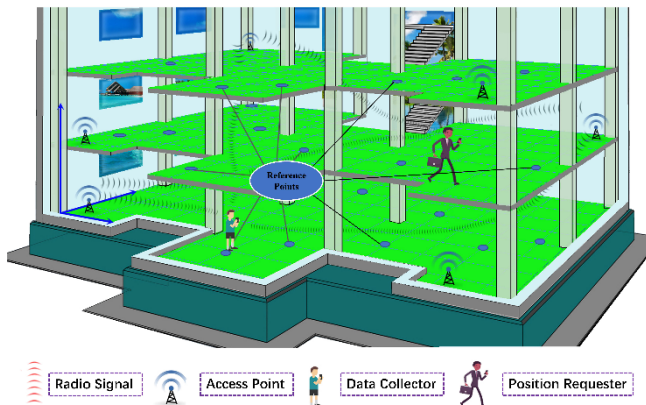




ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

Práctica Final APS – Parte 2: Geolocalización Básica por Potencia WiFi



☀ **Objetivo** A partir de los datos recolectados en la Fase 1, esta segunda parte tiene como objetivo implementar un sistema de geolocalización básica en interiores utilizando como criterio la intensidad de señal (RSSI) de las redes WiFi detectadas. La app Android envía los datos automáticamente en modo localización y el script Python debe estimar la ubicación del usuario (planta y coordenadas X/Y), para posteriormente publicarla en el topic MQTT esperado por la app.

📄 Consolidación de Datos Escaneados (Fase 1)

Durante la Fase 1, cada grupo ha realizado **múltiples escaneos (mínimo 2)** por punto de muestreo. Estos escaneos deben **unificarse y procesarse** para generar una **tabla consolidada de antenas** que se utilizará para estimar la posición en la Fase 2.

🎯 Objetivo

Construir una tabla donde **cada fila representa una antena (BSSID)** y contiene la información necesaria para estimar posición:

- BSSID
- Coordenadas promedio X, Y del punto donde se detectó
- Nivel



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

- Potencia promedio o mediana (opcional para análisis)
- Número de veces que fue detectada (frecuencia)
- Zona estimada

¿Qué hacer con los escaneos repetidos?

Por cada punto de muestreo:

1. Agrupar todos los registros por BSSID.
2. Calcular la **media o mediana** de la potencia RSSI.
3. Registrar las **coordenadas X, Y y nivel** del punto (ya conocidas).
4. Incluir un campo de frecuencia: número de veces que se detectó esa antena.
5. Registrar si es una **antena confiable (fichada)** o no, para filtrar en Fase 2.

Ejemplo de tabla generada:

BSSID	x	y	Nivel	RSSI medio	Frecuencia	Zona
88:96:4D:A4:1F:2A	12.0	8.5	PB	-64.7	3	PB - pasillo
F4:0E:22:11:AA:01	14.0	2.0	P1	-72.5	2	P1 - aula A

Este resumen puede generarse en Python o manualmente en Excel, usando las mediciones guardadas en CSV o SQLite.

Resultado final

La tabla consolidada será la base del fichero base_antenas que usarás en el script de estimación. Solo deben incluirse antenas **con posición fiable** y suficientemente detectadas.

Flujo del Sistema

1. La app envía datos periódicamente en modo localización al topic: `indoor_geolocation/GX/location/data`



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

2. El script Python se suscribe a ese topic, procesa las redes conocidas (antenas previamente geolocalizadas) y estima la posición del usuario basándose en la potencia de la señal.
3. La estimación se publica en el topic:
indoor_geolocation/GX/location/result con el siguiente formato JSON:

```
{ "zona_estimada": "PB - zona central", "x": 12, "y": 1, "nivel": "PB", "timestamp": 1750804956849 }
```

La app suscrita a este topic mostrará la información en pantalla.



◆ Topics MQTT utilizados

- **indoor_geolocation/GX/location/data:** Topic donde la app publica escaneos automáticos en tiempo real. El script debe estar suscrito a este topic para recibir datos continuamente durante la Fase 2.
- **indoor_geolocation/GX/location/result:** Topic al que el script debe publicar la localización estimada del usuario. La app se encuentra suscrita a este topic y muestra la información recibida en su interfaz.



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

Estos topics permiten una arquitectura asíncrona, en la que la app y el servidor pueden comunicarse en tiempo real para estimar y visualizar la posición del usuario dentro del edificio.

📌 Filtrado y uso de datos

- Solo se considerarán antenas (BSSID) cuya posición esté geolocalizada.
- Si no se dispone de geolocalización directa de una antena, se deberá estimar su posición utilizando:
 - Múltiples lecturas desde puntos conocidos.
 - Interpolación de máximos de señal.
 - Asignación visual basada en mapas y cobertura.
- Las antenas capturadas deben filtrarse para quedarse únicamente con las que se tenga seguridad sobre su posición (antenas fichadas).

📅 Base de Datos de Antenas Se debe utilizar una base de datos local (CSV o SQLite) que contenga la ubicación de cada punto de acceso WiFi (antena). Cada entrada debe incluir al menos:

- BSSID (MAC de la antena)
- Coordenadas X, Y
- Nivel (PB, P1, etc.)
- Zona estimada (texto descriptivo opcional)

👤 Filtrado de Señales

- Se descartarán señales con potencia inferior a -85 dBm.
- Solo se tendrán en cuenta antenas conocidas.
- Si no se detecta ninguna antena válida, se podrá devolver "zona_estimada": "desconocida" o no publicar resultado.

🤖 Heurística de Estimación de Ubicación Métodos:



1. Método básico: antena de máxima potencia

- Seleccionar la antena con mayor RSSI y utilizar directamente su información de la base de datos como estimación. La base de datos debe tener registrada su posición exacta (coordenadas X, Y), el nivel y la zona.

```
mejor_antena = max(wifi_scans, key=lambda ap: ap["potencia"])
bssid = mejor_antena["BSSID"]
if bssid in base_antenas:
    info = base_antenas[bssid]
    x, y = info["x"], info["y"]
    nivel = info["nivel"]
    zona_estimada = info["zona_estimada"]
```

- wifi_scans sea una lista de diccionarios con al menos "BSSID" y "potencia" (RSSI en dBm).
- base_antenas sea un diccionario con claves "BSSID" y valores que incluyan "x", "y", "nivel", "zona_estimada".

✓ Análisis paso a paso del código:

```
mejor_antena = max(wifi_scans, key=lambda ap: ap["potencia"])
```

- Se elige la antena con **mayor potencia** (RSSI más cercano a 0), es decir, la señal más fuerte.
- Como los RSSI son negativos (ej. -50, -70), max() selecciona el valor menos negativo, que es correcto.

```
bssid = mejor_antena["BSSID"]
```

- Se extrae la dirección MAC (identificador único) de la antena más potente.

```
if bssid in base_antenas:
    info = base_antenas[bssid]
    x, y = info["x"], info["y"]
    nivel = info["nivel"]
    zona_estimada = info["zona_estimada"]
```

- Se comprueba que esa antena esté **geolocalizada**.



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

- Se recuperan las coordenadas, el nivel y la zona estimada directamente de la base.

✓ ¿Qué devuelve?

- x, y: coordenadas exactas conocidas de esa antena.
- nivel: planta.
- zona_estimada: etiqueta o nombre de zona.

Este método **no requiere ninguna interpolación ni cálculo adicional** y tiene **sentido como aproximación básica** si se presupone que el usuario está más cerca de la antena con mejor señal.

2. Método avanzado (triangulación real):

- Convertir RSSI en distancia usando el modelo de pérdida logarítmica:

$$d = 10^{\frac{A - \text{RSSI}}{10 \cdot n}}$$

donde:

- d: distancia estimada,
- A: RSSI a 1 metro (valor de referencia, por ejemplo -40 dBm),
- n: exponente de propagación (típico entre 2 y 3).

Resolver las coordenadas (x, y) mediante multilateración (por ejemplo, minimizando el error cuadrático entre distancias estimadas y reales).

```
from scipy.optimize import minimize
import numpy as np

def rssi_to_distance(rssi, A=-40, n=2.2):
    return 10 ** ((A - rssi) / (10 * n))

def multilateration(antenas):
    def error(pos):
        return sum((np.linalg.norm(pos - np.array([a['x'], a['y']]))) - a['dist'])**2
        for a in antenas)

    pos0 = np.mean([a['x'], a['y']] for a in antenas), axis=0)
    res = minimize(error, pos0, method='L-BFGS-B')
```



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

```
    return res.x if res.success else pos0
for a in antenas_validas:
    a['dist'] = rssi_to_distance(a['potencia'])

x, y = multilateration(antenas_validas)
nivel = base_antenas[antenas_validas[0]['BSSID']]['nivel']
zona_estimada = estimar_zona_desde_coordenadas(x, y, nivel)
```

Análisis paso a paso del código

```
from scipy.optimize import minimize
import numpy as np
```

✓ Importa funciones necesarias:

- minimize: para optimización.
- numpy: operaciones vectoriales y norm.

Función rssi_to_distance(rssi, A=-40, n=2.2)

```
def rssi_to_distance(rssi, A=-40, n=2.2):
    return 10 ** ((A - rssi) / (10 * n))
```

Qué hace:

- Aplica el **modelo de pérdida logarítmica**:
- Donde:
 - A: RSSI a 1 metro (típico: -40 dBm),
 - n: exponente de atenuación del entorno (2 en aire libre, 2.7-3.5 en interiores).

Función multilateration(antenas)

```
def multilateration(antenas):
    def error(pos):
        return sum((np.linalg.norm(pos - np.array([a['x'], a['y']])) - a['dist'])**2
    for a in antenas)
```

Qué hace:



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

- Define una función de error que calcula, para una posición candidata pos, la **suma de errores cuadráticos** entre:
 - la distancia real al punto (x,y)
 - y la distancia estimada (por RSSI).

Es decir, minimiza

$$E(x, y) = \sum_{i=1}^N \left(\sqrt{(x - x_i)^2 + (y - y_i)^2} - d_i \right)^2$$

```
pos0 = np.mean([[a['x'], a['y']] for a in antenas], axis=0)
```

✦ Punto inicial para el optimizador: el centro geométrico de las antenas conocidas.

```
res = minimize(error, pos0, method='L-BFGS-B')  
return res.x if res.success else pos0
```

- 🔍 Usa el optimizador L-BFGS-B de SciPy (rápido y robusto).
- ✓ Devuelve la posición óptima si converge.

📦 Cálculo completo

```
for a in antenas_validas:  
    a['dist'] = rssi_to_distance(a['potencia'])
```

📦 Añade el campo dist a cada antena válida, estimando su distancia a partir del RSSI.

```
x, y = multilateration(antenas_validas)  
nivel = base_antenas[antenas_validas[0]['BSSID']]['nivel']  
zona_estimada = estimar_zona_desde_coordenadas(x, y, nivel)
```

- ✦ Se estima la posición XY con multilateración.
- ✦ Se asigna el nivel a partir de la primera antena
- ✦ Se deduce la zona con estimar_zona_desde_coordenadas.

✓ ¿Qué devuelve?

- x, y: coordenadas del usuario estimadas mediante multilateración.



ADQUISICIÓN Y PROCESAMIENTO DE LA SEÑAL (APS) GRADO EN INTELIGENCIA ARTIFICIAL

- nivel: planta del edificio.
- zona_estimada: texto tipo "PB - zona central".

⚠ Requiere al menos **3 antenas válidas** con buena señal para una solución robusta.